

Behavioral Synthesis of ICs

Quality Measures

Credentials: majority of information are from the textbook:

Gajski, Daniel D., Nikil D. Dutt, Allen CH Wu, and Steve YL Lin. *High—Level Synthesis: Introduction to Chip and System Design*. Springer Science & Business Media, 2012.

Outline

1. Introduction
2. The Relationship between Structural and Physical Designs
3. Area Measures
 - Datapath
 - Control Unit
4. Performance Measures
 - Electrical Models
 - The Delay of Combinational Circuits.
 - The Delay of Storage Elements
 - System Clock Cycle
5. Other Measures

Objectives



Introduce

Introduce the key steps in transforming a high-level behavioral description into a structural design and the relationship with physical design.



Familiarize

Familiarize with different measures of area within designs and understand how area constraints influence structural design choices.



Examine

Examine the components of a datapath and the role it plays in the execution of instructions within a synthesized design.



Understand

Understand the function of a control unit and its importance in the coordination and control of operations in designs.



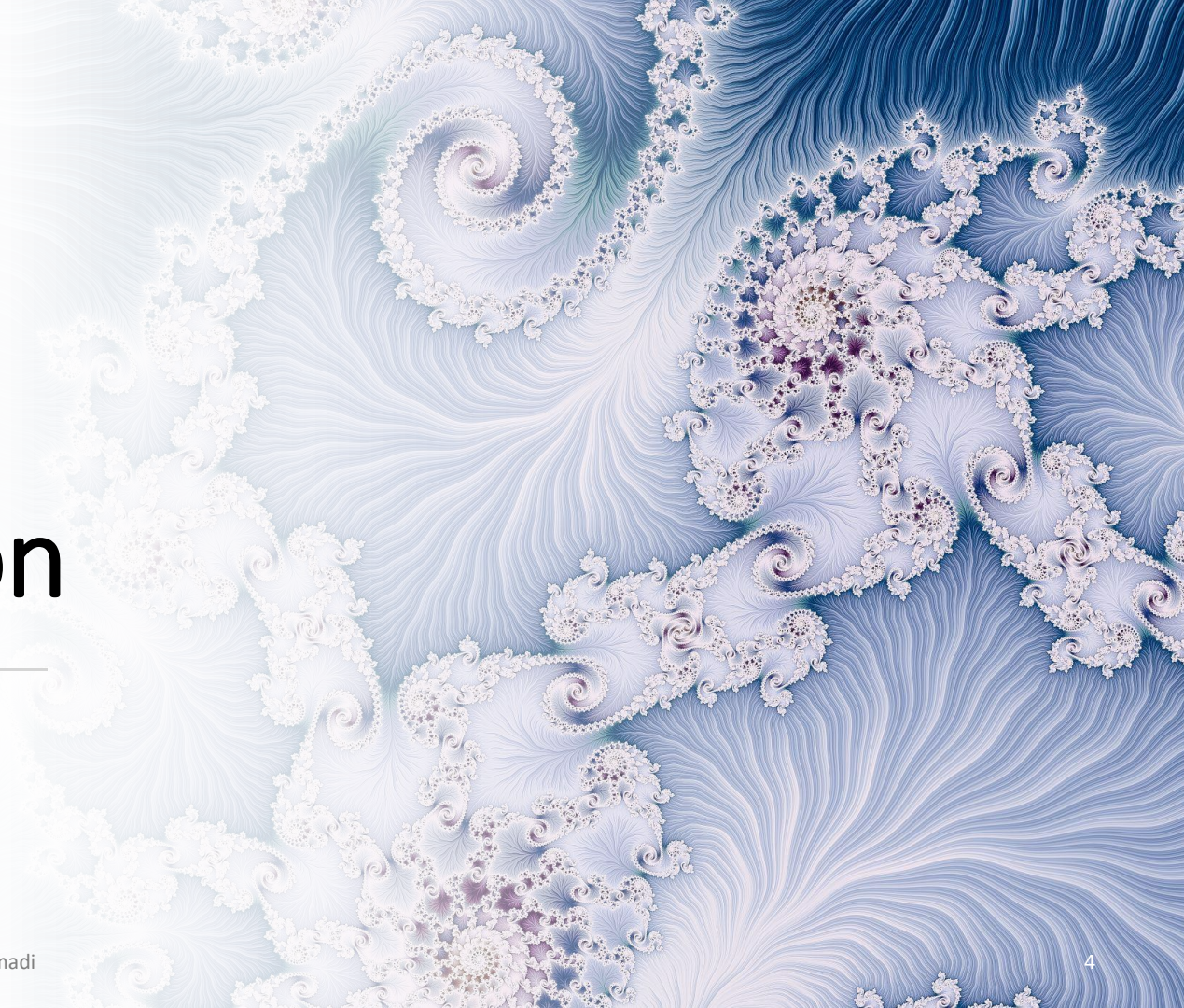
Identify and discuss

Identify and discuss different performance metrics in the context of digital designs and their significance in synthesis.



Delve

Delve into electrical models for components to better appreciate their implications on design constraints and optimization.



Introduction

Introduction

- High-level synthesis generates a structural design that implements the given behavior and satisfies design constraints for area, performance, power consumption, packaging, testing and other criteria.
- Quality measures are needed to support HLS in two ways:
- First, accurate measures are needed to determine the quality of the final synthesized design.
- The quality measures of the final design allow comparison with given constraints and identify the critical spots in the design methodology, tool set or management structure.

Introduction

- For **example**, the total layout area of a chip determines the manufacturing yield, which determines the cost.
- Similarly, a high ratio of metal₁ versus metal₂ wires may point to inferior placement or routing algorithms. Second, good estimates of design quality are needed to guide HLS tools and drive the selection of design style, target architecture and other architectural properties.
- For **example**, if the **clock period** is too long a **nonpipelined multiplier** must be replaced with a **pipelined multiplier**.
- Similarly, a design with **three buses** requires more **area** than a design with **two buses**. Thus, different quality measures are used to evaluate the quality of different design alternatives during partitioning, scheduling and allocation in HLS.

Introduction

- There are typically **three major measures** used to support design decisions: **size**, **performance** and **power consumption**.
- Size measures deal with the physical size of a design.
- Performance measures are related to the propagation delay through a design and the total execution time.
- Power measures represent the power dissipation across a design and indicate hot spots in the implementation.
- There are also other measures used to evaluate the design quality with respect to **verifiability**, **testability**, **reliability** and **manufacturability**.
- Since all design decisions depend on these measures, their accuracy and fidelity are essential for generation of high-quality designs.



Structural vs. Physical Designs

Structural vs. Physical Designs

- **High-level synthesis** transforms an input **behavioral description** into a **structural design** composed of a datapath netlist and a control unit.
 - The datapath netlist consists of a set of generic register-transfer (RT) components (e.g., functional, interconnected and storage units).
 - The control unit specifies the control signals for executing register transfers in each state, as well as the sequencing for the design's next state.
- The process of generating manufacturing data for custom or semi-custom technologies from a structural design consists of several steps including **technology mapping**, **module generation**, **floor planning**, **placement** and **routing**.
- Technology mapping assigns real components from a physical library to the generic components in the structural design. It is usually followed by some optimization procedures to reduce the total area and delay of a design.
- We also assume that the technology mapper selects different layout styles for different parts of the design.

Structural vs. Physical Designs

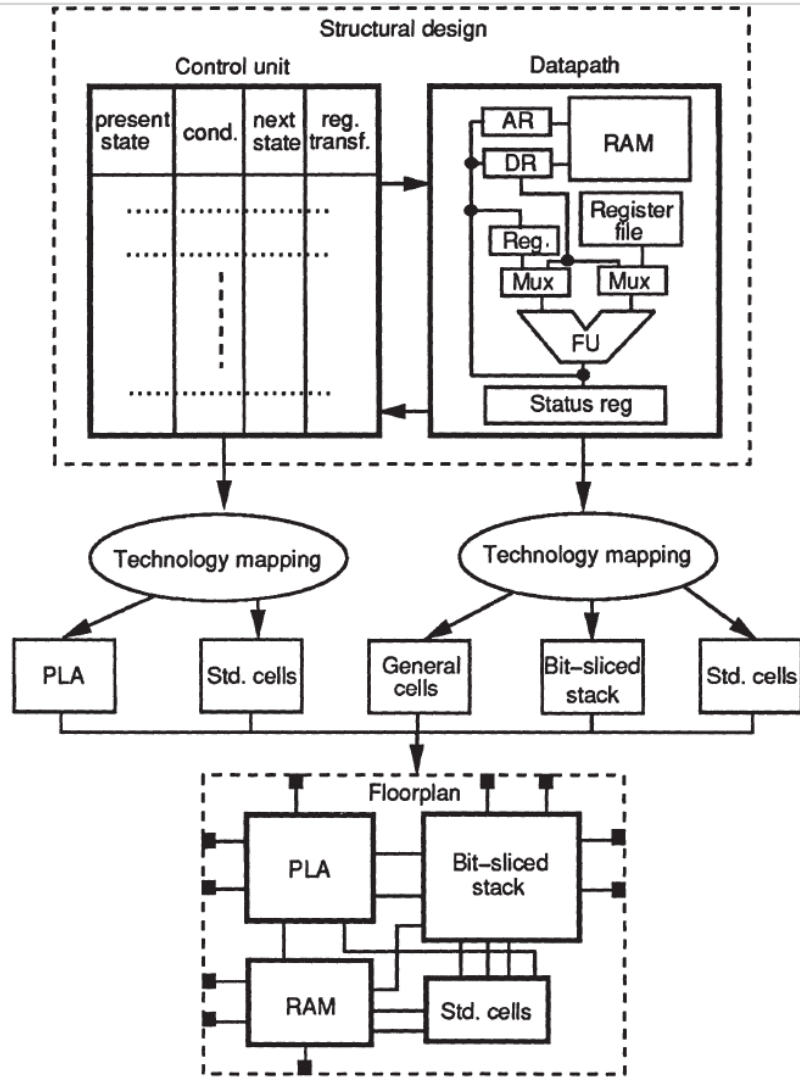
- For **example**, regularly structured units, such as **adders**, **subtractors**, **ALUs**, **multiplexers** and **registers**, can be mapped into bit-sliced stacks, general cells or standard cells; the control-state table can be mapped into a PLA or standard cells; and storage units, such as RAMs, ROMs and register files, as well as **functional units**, such as multipliers, are mapped into **macro cells**.
- The design may contain a mixture of arbitrarily sized bit-sliced stacks, general cells, macros and standard-cell blocks, which we simply refer to as "modules".
- Module generators perform floor planning and routing of each module independently.
- The chip floor planner determines the positions and interconnections of modules and generates the chip layout.

Structural vs. Physical Designs

- The **total area** of a design is the sum of the area of its **modules**, **I/O pads** and **pad drivers**, the **chip routing areas** and the remaining **wasted areas**, as illustrated in Figure 1.
- Since macros are predesigned, their areas and shapes can be obtained directly from component libraries.
- However, the areas and shapes of datapath and control modules mapped into **bit-sliced stacks**, **standard cell blocks** or **PLA macro cells** vary greatly depending on their layout styles and architectures.

Structural vs. Physical Designs

- **Figure 1:** The relationship between structural and physical designs.





Area Measures

Area Measures

- Traditional **area measures** divide the physical design into **active-unit** area and **interconnection area**.
 - **Active-units** include functional units, such as ALU s, adders and multipliers, and storage units, such as registers, register files, ROMs and RAMs.
 - **Interconnection units** include multiplexers, buses and wires.
- Various methods have been used to measure active-unit areas.
 - One method uses the **number of gates** (AND, OR and NOT) in the Boolean expressions describing a functional unit.
 - Another measure approximates the active-unit area with the **sum of cell areas** where each cell implements one symbol in the **schematic diagram**. This cell area is obtained from **cell library data-books** provided by manufacturers.
 - The area can also be approximated by the **number of transistors** multiplied by a transistor density-coefficient in $\mu m^2/transistor$ obtained by averaging layout area per transistor over all cells available in the library or over some sample real designs.

Area Measures

There are some commonly used methods for **wire length estimation** in IC design:

- **Rent's Rule:** Rent's rule is an empirical formula that relates the number of **external signal connections** (P) to the **number of gates** (G) with a relationship $P = tG^p$, where t is the **Rent coefficient** and p is the **Rent exponent**. This rule is often used to estimate the mean interconnection length.
- **Statistical Wire Length Models:** These models use **statistical distributions** that depict common wire length patterns in a layout. For instance, a **Gamma distribution** can be employed to approximate the **distribution of wire lengths in a typical circuit design**, which can then be used to calculate the average wire length.
- **Placement and Routing Algorithms:** These algorithms actually place the components and route the connections in a design while aiming to minimize wire lengths as much as possible, given certain constraints. They can provide exact wire lengths after the design is completed.
- **Half-Perimeter Wire Length (HPWL):** For a quick estimate, designers often use the Half-Perimeter Wire Length, which can be **estimated during the placement phase**. HPWL is a bounding box estimation tool that calculates the half-perimeter of the smallest rectangle that can enclose all the terminals of a net.
- **Probabilistic Models:** Probabilistic estimates like **Monte Carlo simulations** can be used to randomly generate and analyze multiple wiring possibilities to obtain a **statistical estimate** of the expected wire lengths.

Area Measures

- With regard to **interconnection area**, most metrics assume that the layout area is directly proportional to the number and size of some interconnection units.
- Hence, the **number of multiplexers** or equivalent 2-to-1 multiplexers (MUX-2) is a common measure for the interconnection area in datapath-allocation algorithms.
- Likewise, the **number of multiplexer inputs** and the number of wires are common measures for the **routing area**.
- Generally speaking, VLSI circuit area estimation methods are mostly based on a combination of geometry and statistics!

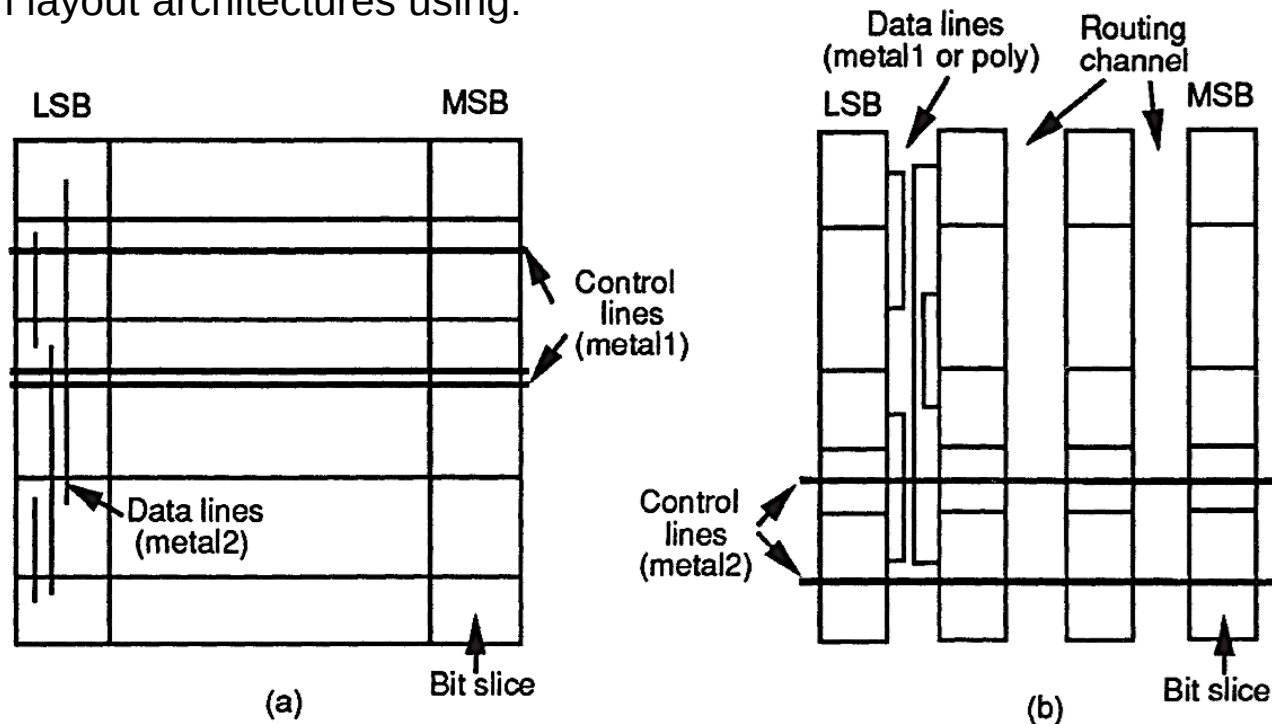
Area Measures: Datapath

- A **datapath** consists of a set of regularly structured RT components, such as ALUs, multiplexers, latches, drivers and shifters.
- Datapath layout is accomplished with a stack of **functional** and **storage** units that are placed one above the other.
- Each unit consists of **bit-slices** and all units have bit slices of the same width. However, bit slices in different units may be of different height. All bit slices are aligned starting with the least significant bit (LSB) and distinct units are stacked on top of another.
- The stack grows **horizontally** when the **bit width** increases, and it grows vertically when the number of units increases (Figure 3.a).
- Each bit slice of a unit may be a handcrafted custom cell or may be implemented with one row of connected standard cells as shown in Figure 2.a and (b), respectively.
- The difference between **custom** and **standard-cell** styles is in the layers used for routing of control and data wires, use of custom or standard cells and routing of data lines over the cells or in a separate channel.

Area Measures: Datapath

→ **Figure 2:** Two data path layout architectures using:

- a) custom cells,
- b) standard cells.

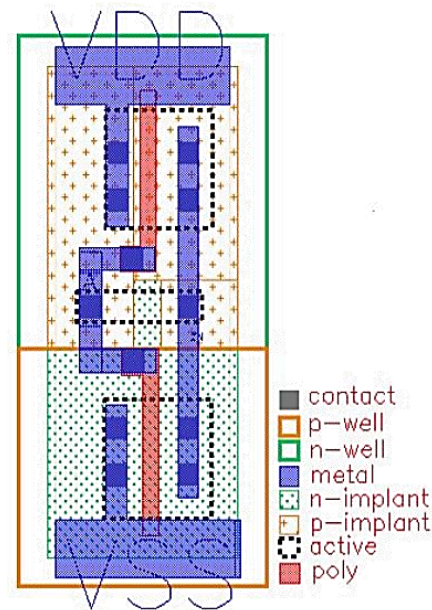


Area Measures: Datapath

- In the first layout architecture (Figure 3.b), diffusion strips for P and N transistors are placed horizontally.
- **Power** and **ground wires** run horizontally in the first metal layer .
- The **control lines** common to different bit slices in each unit run also horizontally in the first metal layer.
- **Data lines** connecting distinct units in each bit slice run vertically in the second metal layer.

Area Measures: Datapath

- In the second layout architecture (Figure 3.c), a bit slice of each unit consists of one or more standard cells.
- **P** and **N diffusion strips** are placed vertically.
- **Power** and **ground wires** run vertically in the first metal layer .
- **Control lines** run over the standard cells in the second metal layer.
- **Data lines** are placed in the routing channel and run vertically in the first metal or the polysilicon layer.
- The **connections between standard cells** inside each bit slice is also placed in the routing channel.



Inverter

Area Measures: Datapath

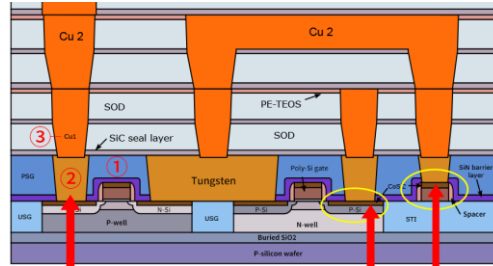
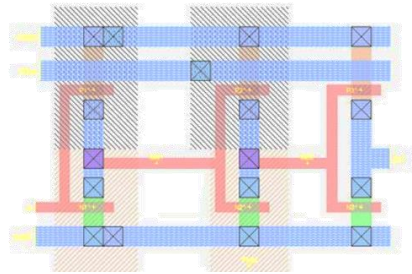
- To compute the height (H_{dp}) of a **bit slice** (Figure 3.a), we observe that H_{dp} is proportional to the number of transistors in the bit slice. Each bit slice of a unit in Figure 3.c consists of several **diffusion strips** separated by gaps.
- The **transistors** on each diffusion strip are separated by metal-diffusion contacts or by the minimal poly-to-poly spacing. Thus, the width of *Unit* (W_{unit}) in Figure 3.c can be computed as a product of the number of transistors ($tr(unit)$) and the transistor-pitch coefficient (α) in $\mu m/transistor$. α is obtained by averaging the ratio of cell width and the number of transistors per cell over all units in the library.
- Thus,

$$W_{unit} = \alpha \times tr(unit)$$

- Consequently, the height of the bit-sliced stack of n units is

$$H_{dp} = \sum_{i=1}^n W_{unit} = \alpha \times \sum_{i=1}^n tr(unit_i) \quad (2)$$

VLSI Layout (a basic view)

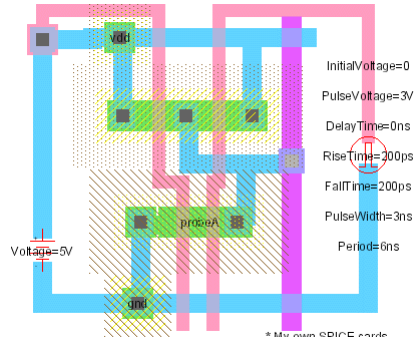


Uses of Tungsten When Contacts are Formed

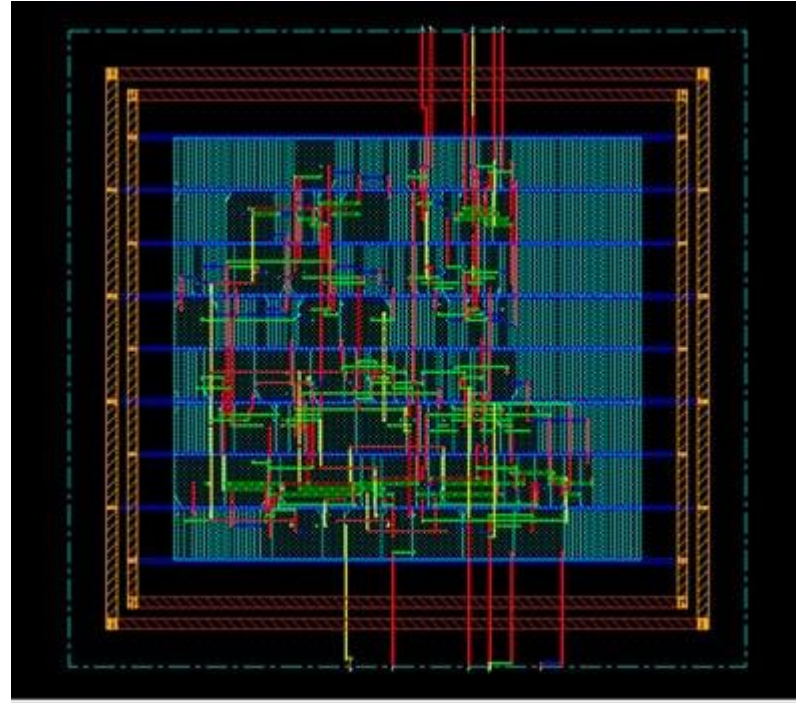
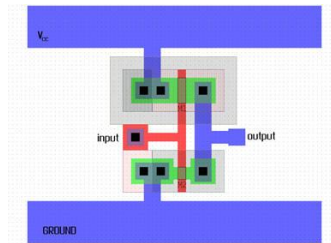
- When a contact point is needed in a deep area
- When heat treatment is required following processes (1) and (2), but substance (3) has weak resistance to heat

Location and role of a barrier metal

- Reducing the resistance of contact points

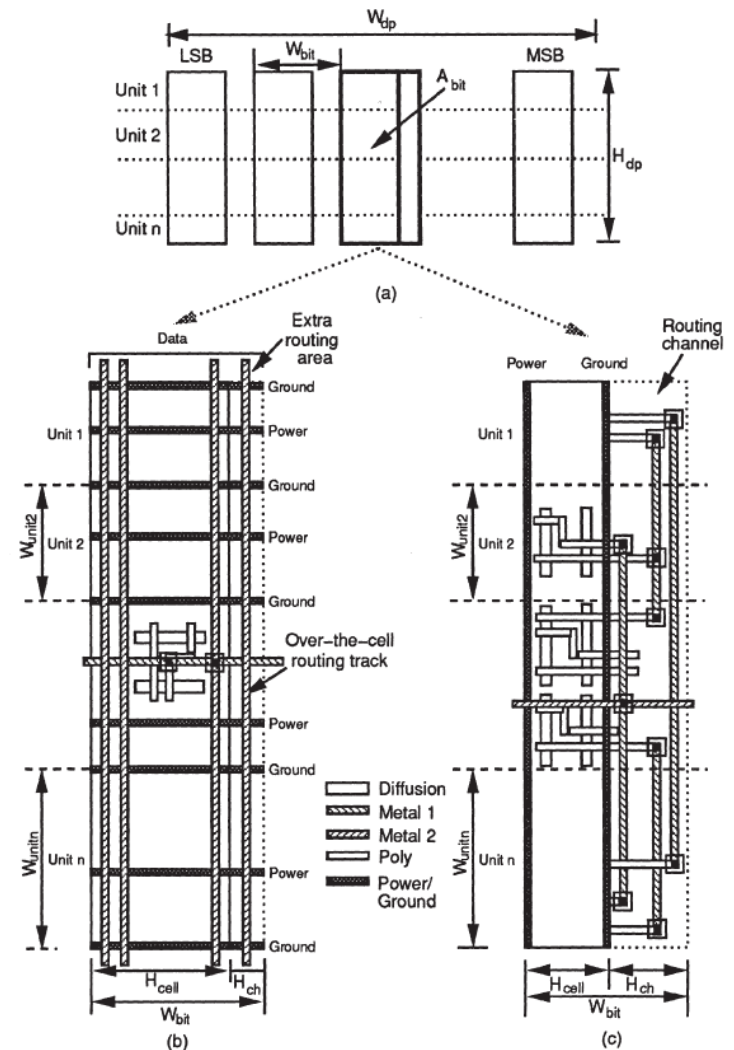


* My own SPICE cards
* That get stuffed into the deck



Area Measures: Datapath

- **Figure 3:** The layout models: (a) datapath stack, (b) custom cell architecture, (c) standard cell architecture.



Area Measures: Datapath

- The Equation 2 will hold for **standard-cell architecture** even if each bit slice is implemented with two or more rows of standard cells.
- A different coefficient α' must be used in that case. Thus, the height of the bit-sliced stack of n units with an m -row implementation is

$$H_{dp} = \frac{\alpha' \times (\sum_{i=1}^n W_{unit})}{m}$$

- Similar assumptions can be made for custom-cell architecture shown in Figure 3.b.
- Although the P and N strips are placed horizontally in several rows, W_{unit} can be computed by Equation 2 using a different transistor-pitch coefficient α'' .
- This assumption holds because the **height of the unit slice** (H_{cell}) is a constant and the **unit width** (W_{unit}) thus must reflect the size of the cell in number of transistors.

Area Measures: Datapath

- The **width** W_{bit} of a bit slice is equal to the sum of the **height of the unit slice** (H_{cell}) and the height of the **routing channel** (H_{ch}).
- For both layout architectures H_{cell} is a constant since all unit slices are pre designed to be of the same height.
- H_{ch} is calculated as a product of the wire pitch (β), and the difference between the number of **estimated routing tracks** (Trk_{est}) required to completely connect all nets in one bit slice and the number of **available over-the-cell routing tracks** (Trk_{top}). Thus,
$$H_{ch} = \begin{cases} 0; & \text{if } Trk_{top} \geq Trk_{est} \\ \beta \times (Trk_{est} - Trk_{top}) & \text{if } Trk_{top} < Trk_{est} \end{cases}$$
- where Trk_{top} in standard-cell architecture is equal to zero, and coefficient β is equal to the sum of the minimal wire width and the minimal spacing between two metal wires.

Area Measures: Datapath

- An estimate for the required **number of tracks** in each bit slice can be obtained only after the position of each unit in the bit slice is determined.
- A **fast algorithm** with pseudo linear time complexity, such as the min-cut algorithm can be used for this purpose.
- The **required number of tracks** can be estimated by the **maximal density** which is defined as the maximum number of connections across any cut perpendicular to the channel.
- A better estimate can be obtained by using some **simple routing algorithms**, such as the **left-edge algorithm** which has $O(n \log(n))$ complexity where n is number of nets.
- Thus, the datapath area (A_{dp}) can be calculated as a product of the number of bits (b_w) and the area of one bit slice, i.e.,

$$A_{dp} = b_w \times H_{dp} \times (H_{cell} + H_{ch})$$

Area Measures: Datapath

- This equation gives an upper bound on the datapath area.
- The **bound** is **proportional** to the product of the **number of transistors** and the **number of routing tracks**.
- The **number of transistors** can be approximated from the Boolean expressions describing each unit slice or counted from its schematic.
- The **number of tracks** can be approximated by the track density after a linear placement.
- Better estimates can be achieved with **algorithms of higher complexity**.
- Since the **number of components** in the **datapath** is small, those more accurate estimates are not necessarily computationally intensive.

Area Measures: Control Unit

- The **control unit** in a **FSMD** model can be described by the control **state table** which specifies the next-state and **control signals** as a function of present states and conditional or status signals in a tabular form.
- Figure 4.a shows a sample control state table with the input present-state and conditional/status signals and the output next-state and control signals.
 - The **present states** are encoded as binary values $P_k \dots P_1 P_0$, where $k \geq \log_2(m) - 1$ and m is the number of states.
 - The **next states** are encoded as binary values $r_k \dots r_1 r_0$.
 - Each **output** c_i controls a functional, storage or interconnection components in the datapath, whereas the outputs r_j specify the next state.

Area Measures: Control Unit

- **Figure 4:** Control unit description:
- (a) state table,
- (b) Boolean equations for output signals,

Input						Output			
$I_1 \ I_2 \ I_3 \ I_4 \ I_5$						$O_1 \ O_2 \ O_3 \ O_4$			
Present state		Conditions/ status				Next state		Control signals	
P_1	P_0	S_2	S_1	S_0		r_1	r_0	C_1	C_0
State 1	0	1	1	0	0	1	0	0	1
State 2	1	0	1	0	1	1	0	1	0
State 3	1	0	0	0	1	1	1	1	1

(a)

$$O_1 = (I_1' I_2 I_3 I_4' I_5') \text{ OR } (I_1 I_2' I_3 I_4' I_5) \text{ OR } (I_1 I_2' I_3' I_4' I_5)$$

$$O_2 = (I_1 I_2' I_3' I_4' I_5)$$

$$O_3 = (I_1 I_2' I_3 I_4' I_5) \text{ OR } (I_1 I_2' I_3' I_4' I_5)$$

$$O_4 = (I_1' I_2 I_3 I_4' I_5') \text{ OR } (I_1 I_2' I_3' I_4' I_5)$$

(b)

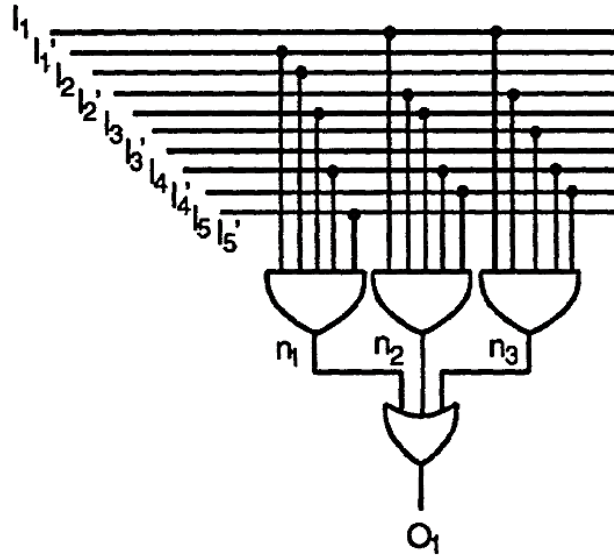
Area Measures: Control Unit

- The **control unit** consists of a **state register** and **control logic**.
- There are two commonly used techniques for implementation of control units:
 - **standard cells** and
 - **programmable logic arrays** (PLA) as described in last lecture.
- In general, area estimates can be performed for standard-cell and logic array implementations based on **sum-of-product** expressions of next-state and control signals (Figure 4.b).
- **Note:** in reality, a number of optimization procedures are applied in order to reduce the size of the control logic.
- In a basic evaluation, the impact of optimization is ignored to find an upper bound on the control-unit area.

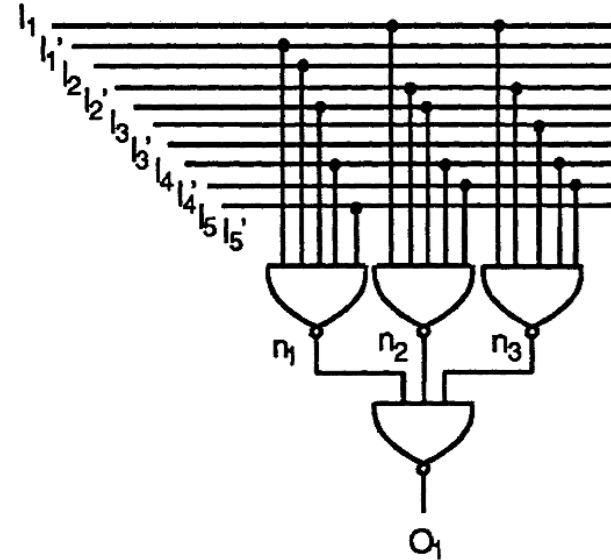
Area Measures: Control Unit

→ **Figure 4:** Control unit description (continues):

- c) two-level AND-OR implementation,
- d) two-level NAND-NAND implementation,



(c)



(d)

Area Measures: Control Unit

- To simplify **area estimates** for **standard cell implementations**, we make a number of assumptions.
- We assume that a product term in the **sum-of-product expression** for each output signal includes each present-state signal and (in the worst case) each conditional/status signal; these inputs to the product term may be complemented.
- We also assume that each product term is implemented by an AND gate and the sum-of-products by an OR gate, as illustrated in Figure 4.c for output O_1 .
- presumably, we can replace the AND-OR implementation with an equivalent NAND-NAND implementation as shown in Figure 4.d.

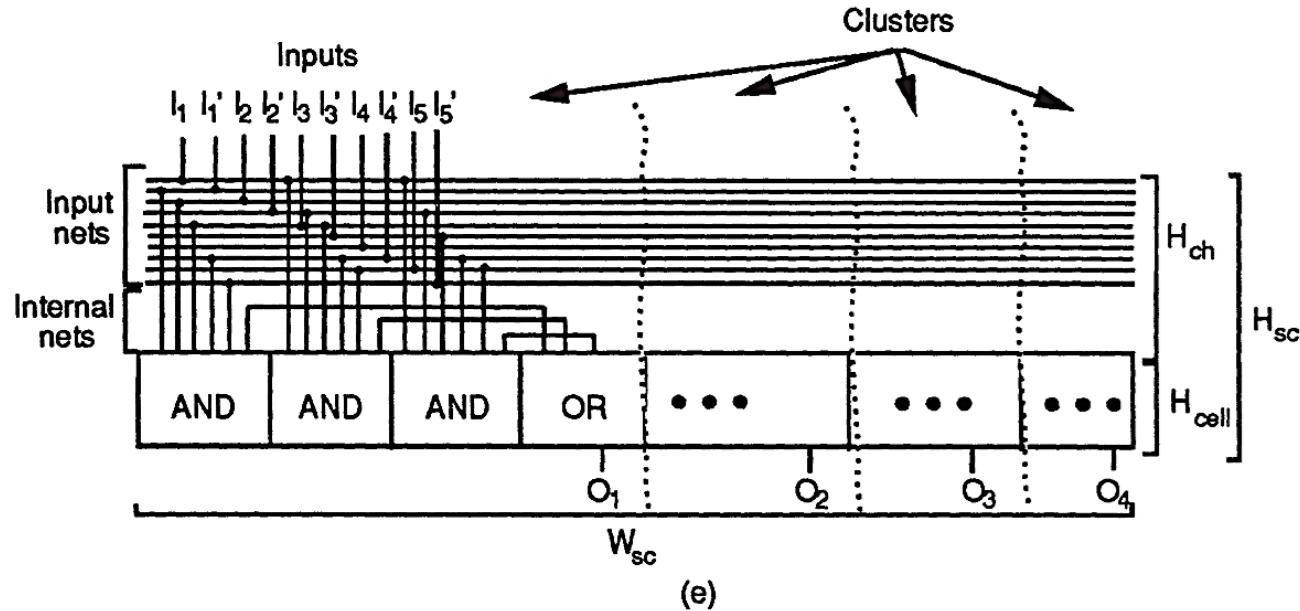
Area Measures: Control Unit

- We can assume that all the gates for implementation of the **control unit** are placed in a **single row of cells**, the inputs appear at the top, and the outputs appear at the bottom, as shown in Figure 4.e.
- We also assume that all the gates needed for the implementation of an output signal are clustered together as shown for signal O_1 in Figure 4.e.
- This requirement for strong clustering prevents sharing of an AND gate between two expressions with the same product term.
- The **layout area of the control unit** using the **standard-cell implementation** (A_{sc}) is then equal to the product of **width** (W_{sc}) and **height** (H_{sc}), that is,
$$A_{sc} = W_{sc} \times H_{sc} \quad (3)$$
- where W_{sc} is proportional to the number of transistors and H_{sc} is proportional to the number of routing tracks. The number of transistors can be computed from the sum-of-product expression for each output signal.

Area Measures: Control Unit

→ **Figure 4:** Control unit description (continues):

e) standard cell layout style.



Area Measures: Control Unit

- In **CMOS technology**, as an example, each n -input AND or OR gate has $2n + 2$ transistors.
 - **Note** that n -input NAND and NOR gates need only $2n$ transistors.
- Since each product term in a **sum-of-product expression** is implemented with one AND gate and one OR gate (e.g., Figure 4.c), we can compute the required number of transistors.
- Each literal in a product term contributes 2 transistors, each product term contributes 2 transistors in the AND gate and 2 transistors in the OR gate and the OR gate contributes an additional 2 transistors.
- Let $occur(O_i)$ and $term(O_i)$ be the number of occurrences of literals and number of terms in the sum-of-product expression of the signal O_i , respectively.
- Let $tr(Reg)$ be the number of transistors in one-bit state register. Thus, for a AND-OR implementation of the control unit the width of the control unit is computed as
$$W_{SC} = \alpha \times \left(\left(\sum_{i=1}^{\lceil \log_2 m \rceil + n} (2term(O_i)occur(O_i) + 4term(O_i) + 2) \right) + (\lceil \log_2 m \rceil \times tr(Reg)) \right) \quad (7)$$
- where m is the number of states and n is the number of control signals and α is the transistor-pitch coefficient

Area Measures: Control Unit

- The height H_{sc} is computed as the sum of the standard-cell height (H_{cell}) and the channel height (H_{ch}).
- H_{ch} is proportional to the number of tracks used by input signals and internal nets connecting AND and OR gates.
- We assume that each input signal requires two tracks for the true and complemented values. We assume that there are $\lceil \log_2 m \rceil$ **state signals** and C **conditional/status signals**.
- The maximum number of tracks required for routing internal nets in each cluster is equal to the number of terms in the sum-of-product expression for a particular output signal.
- Since clusters do not overlap, the maximum number of required tracks to route all internal nets is equal to the largest number of terms used in any particular sum-of-product expressions of an output signal. Thus, the height of the control unit H_{ac} is

$$H_{sc} = H_{cell} + \beta \times (2(\lceil \log_2 m \rceil + C) + MAX(\bigvee_{i=1}^{\lceil \log_2 m \rceil + n} term(O_i))),$$

- where β is the wire pitch in the channel.



Performance Measures

Performance Measures

- Traditionally, design **performance** for HLS has been characterized by such measures as the **clock frequency**, the **number of instructions** or the **number of additions/multiplications executed in a second**.
- Interestingly, these individual measures do not characterize the actual design performance with respect to a particular description.
- Instead, the performance of a design for a particular description is measured by the **total time needed to execute that description**.
- Since a description may contain **loops** without fixed bounds and "**if**" statements in which "**then**" and "**else**" **branches** require different amounts of time, the execution time of such a description will **depend on the input data**.
- Thus, the execution time for any description is equal to the product of the number of control steps needed to execute the description and the duration of the control step (usually a clock period).
- Hence, **minimizing the clock period** always improves the performance for a fixed schedule.
- Performance can also be improved by minimizing number of control steps on the critical path through the computation, using more resources and executing in parallel whenever possible.

Performance Measures: Electrical Models

- The lumped RC model, also called the **Elmore delay** model, is widely used for delay calculation.
- In the lumped RC model, the propagation delay along a path from the start point to the end point, $(t_p(start, end))$, is computed as a product of lumping all of the resistances R_j and capacitances C_k along the path, that is,

$$t_p(start, end) = \sum_j R_j \times \sum_k C_k \quad (10)$$

- This equation can be used to obtain **delay of a connecting wire** between two components as shown in Figure 7.a.

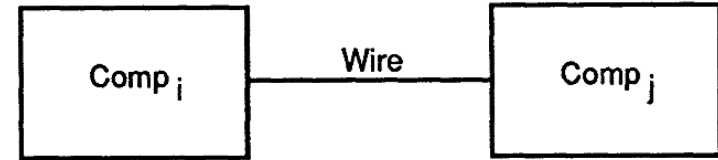
Performance Measures: Electrical Models

→ **Figure 7: Wire:**

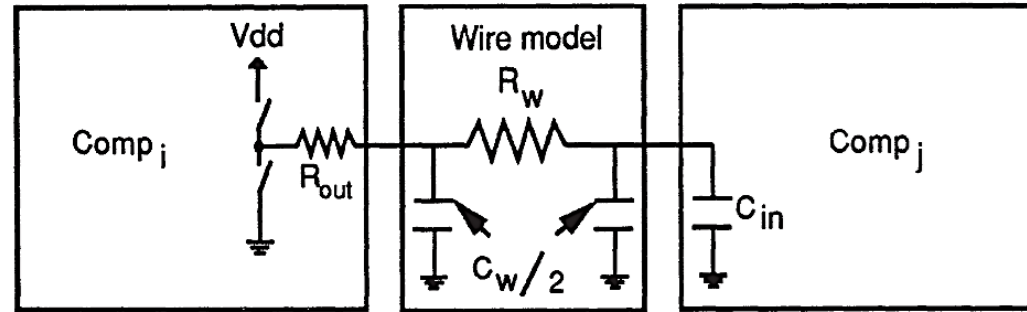
- a) RT model,
- b) equivalent RC delay model.

→ In CMOS technology we model a component with its input **capacitance** (C_{in}) and its output **resistance** (R_{out}), as shown in Figure 7.b.

→ For the **connecting wire**, we use the well known **π -model** with input capacitance ($C_w/2$), wire resistance (R_w) and the output capacitance ($C_w/2$).



(a)



(b)

Performance Measures: Electrical Models

- Since a wire is a thin sheet of metal of fixed thickness defined by the fabrication process consisting of rectangular segments, the wire resistance is equal to the product of the sheet resistance (R_s) in Ω/square and the ratio of the wire length (L_w) and wire width (W_w), that is,

$$R_w = R_s \left(\frac{L_w}{W_w} \right) \quad (11)$$

- The wire capacitance (C_w) is equal to the product of the wire area and the ratio of the dielectric constant (ϵ) and the wire thickness (t), that is,

$$C_w = (L_w W_w) \left(\frac{\epsilon}{t} \right) \quad (12)$$

- Using Equations 11 and 12, we can compute the **propagation delay** ($tp(net_k)$) of a wire net net_k used by a component ($comp_i$) to drive load components ($comp_j$ $1 \leq j \leq n$) as

$$t_p(net_k) = (R_{out}(comp_i) + R_w) \left(C_w + \sum_{j=1}^n C_{load}(comp_j) \right) \quad (13)$$

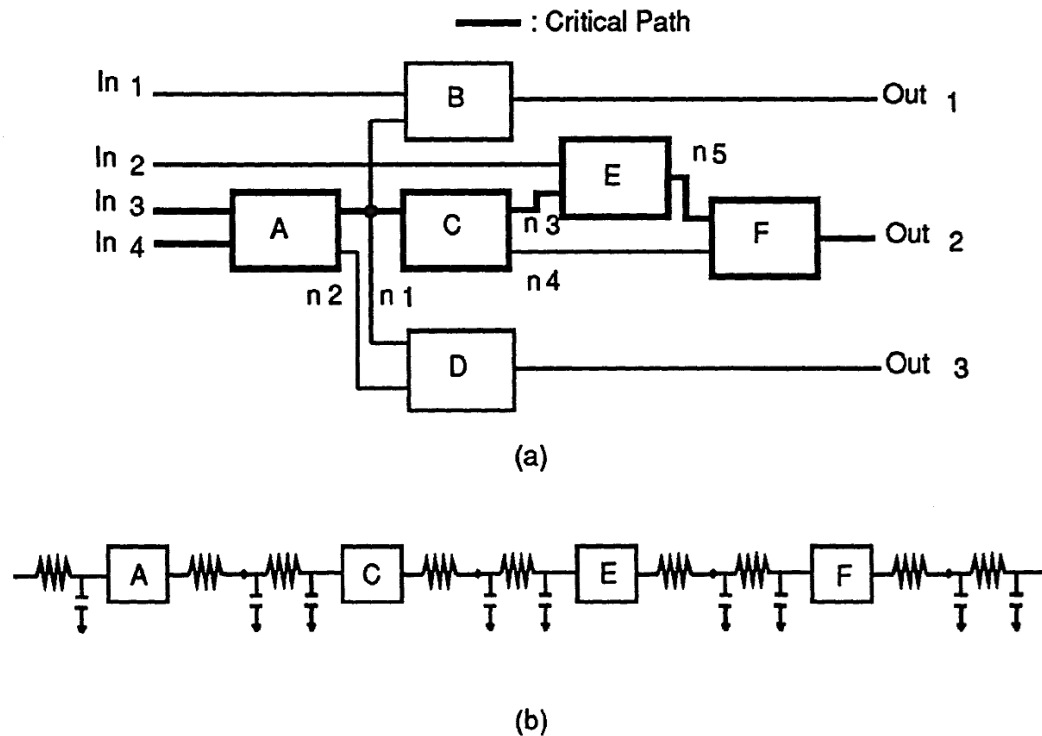
Performance Measures: The Delay of Combinational Circuits

- Each **combinational circuit** consists of one or more components.
- The delay of a combinational circuit is the **propagation delay** on the **critical path** through the circuit, that is, the longest delay from any input to any output.
- For **example**, for the circuit in Figure 8 the critical path is from input I_3 to the output O_2 .
 - The critical path goes through components A , C , E and F and nets n_1 , n_4 , n_6 , n_7 and n_8 .
 - Thus, the delay on a path from an input port to an output port is equal to the sum of propagation delays of all components and all the nets on the critical path.
- Let $COMP(I,O)$ and $CONN(I,O)$ define all the components and all the nets on the path from an input port I to an output port O . Thus, the **propagation delay** ($t_p(I,O)$) from the input port I to the output port O is

$$t_p(I, O) = \sum_{comp_i \in COMP(I, O)} t_p(comp_i) + \sum_{net_i \in CONN(I, O)} t_p(net_i) \quad (14)$$

Performance Measures: The Delay of Combinational Circuits

→ **Figure 8:** The critical path in a combinational circuit.



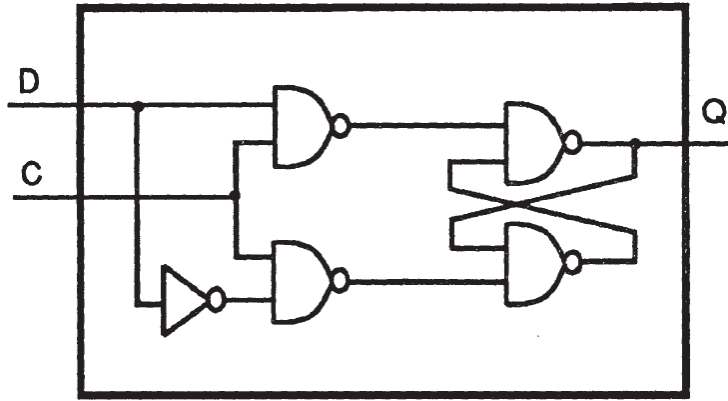
Performance Measures: The Delay of Storage Elements

- **Storage elements** are typically composed of **latches** and **flip-flops**.
- The simple D-type latch is shown in Figure 9.a. As shown in figure 9.b, we can identify two different **delay paths** through this D-type latch:
 - The **data-to-output delay** (t_{DQ})
 - The **clock-to-output delay** (t_{CQ}),
- From the latch's schematic in figure 9.a, we can also conclude that $t_{DQ} > t_{CQ}$ since an extra inverter is on the data-to-output path.
- In order to ensure proper operation of the latch, two **timing constraints** are imposed:
 - The **setup time** (t_{setup})
 - The **hold time** (t_{hold}).
- The setup and hold times specify time intervals before and after clock transitions during which a data signal must be stable to ensure proper operation of the latch (figure 9.b).
- It is obvious from figure 9 that $t_{DQ} = t_{setup} + t_{CQ}$.

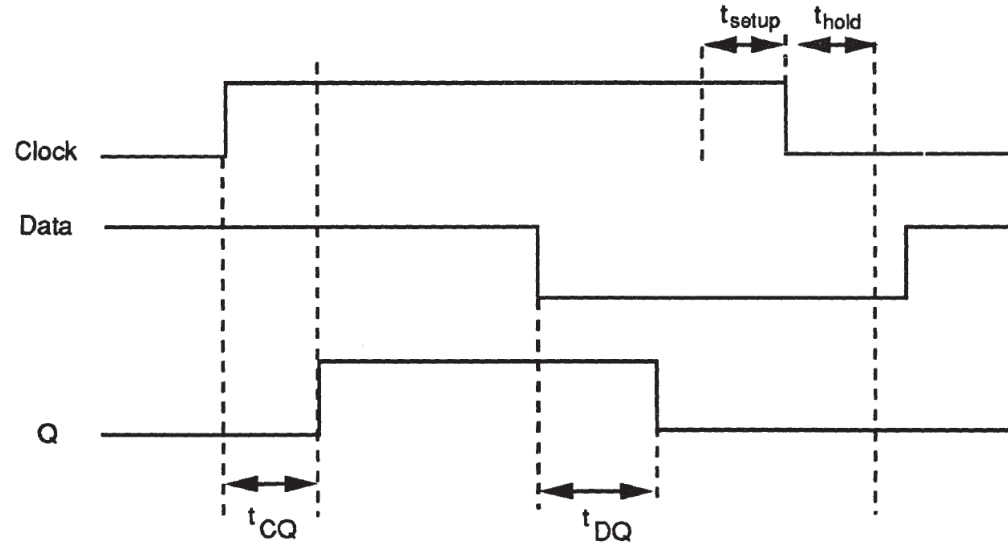
Performance Measures: The Delay of Storage Elements

→ **Figure 9:** D-latch:

- a) logic schematic,
- b) delays and constraints.



(a)



(b)

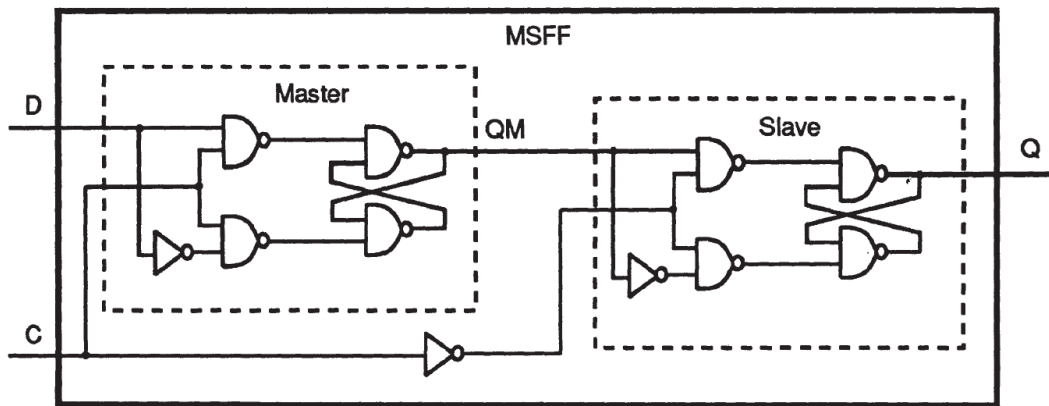
Performance Measures: The Delay of Storage Elements

- **Master-slave** (MS) and **edge-triggered** (ET) flip-flops respond to the transition of the clock signal.
- In a MS flip-flop the master latch is active when the clock signal is high while the slave latch is active when the clock signal is low (Figure 10.a).
 - The output of the **MS flip-flop changes** after the **high-to-low transition** of the clock and the **propagation delay** (t_{CQ}) of the slave latch. Thus, the propagation delay of the MS flip-flop is equal to t_{CQ} of the slave latch.
 - Since the **data signal** must propagate through the master latch before a high-to-low clock transition and still satisfy t_{setup} of the slave latch,
$$t_{setup}(MSFF) = t_{DQ}(Master) + t_{setup}(Slave).$$
- **Note** that the master latch (and hence the MSFF) is not sensitive to changes in the input data while the clock is low.
- For **example**, the **glitch** in D while clock is low does not affect the output Q of the MSFF (Figure 10.b). Hence the hold time for the MS flip-flop is equal to the hold time (t_{hold}) of the master latch.
- We can formulate similar definitions for an ET flip-flop which propagates data on the low-to-high transition of the clock signal.

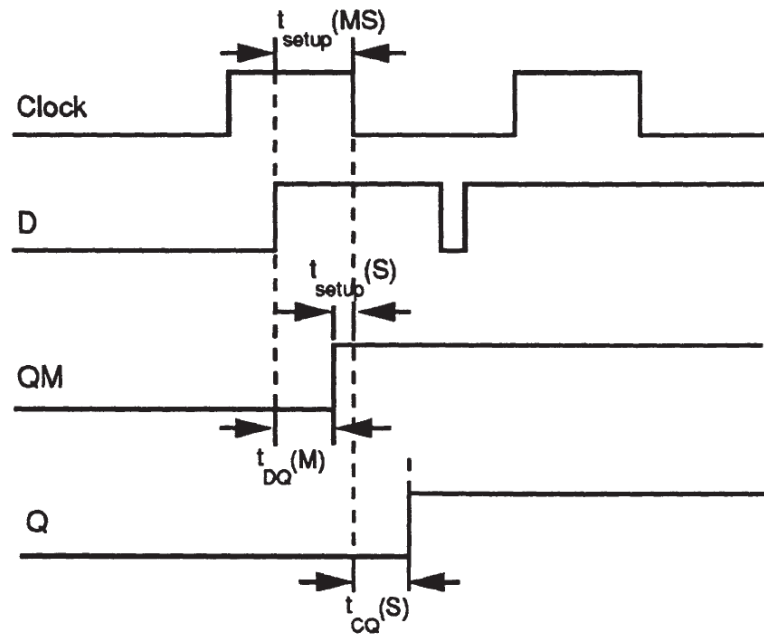
Performance Measures: The Delay of Storage Elements

→ **Figure 10:** Master-slave flip-flop:

- a) D-latch implementation,
- b) Delays and constraints.



(a)



(b)

Performance Measures: System Clock Cycle

- Since the **execution time** of a description is equal to the product of the **number of control steps** and the **clock period**, number of control steps is good measure of performance for a fixed clock period.
- Similarly, **clock period** is a good quality performance measure for a scheduled behavioral description with a **fixed number of states**.
- The **clock cycle** is determined by the **worst register-to-register delay** in the FSM architecture.
- For **example**, a typical register-to-register delay in a datapath is shown in Figure 11. It includes the delay through the **source registers**, **ALU** and **connections**, and the **setup time** of the destination register. In other words, we can specify the clock period by

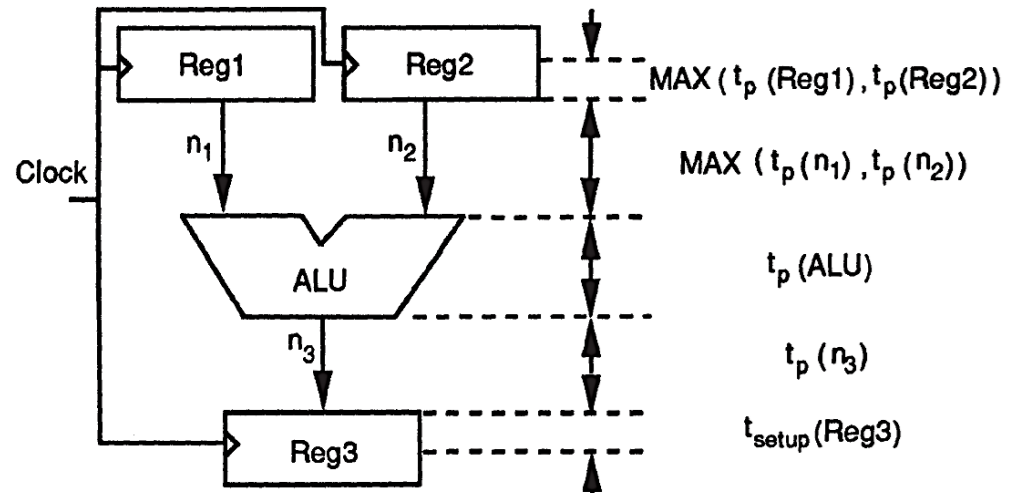
$$t_{clock} = MAX(t_p(Reg_1), t_p(Reg_2)) + MAX(t_p(n_1), t_p(n_2)) + t_p(ALU) + t_p(n_3) + t_{setup}(Reg_3) \quad (15)$$

Performance Measures: System Clock Cycle

→ **Figure 11:** The register-transfer path.

→ **Note:**

- If the data transfer has more components, Equation 15 must be adjusted approximately.
- For **example**, if a multiplexer component is inserted at some input port, an extra multiplexer propagation-delay and the corresponding connection delay must be added to the Equation 15.



Performance Measures: System Clock Cycle

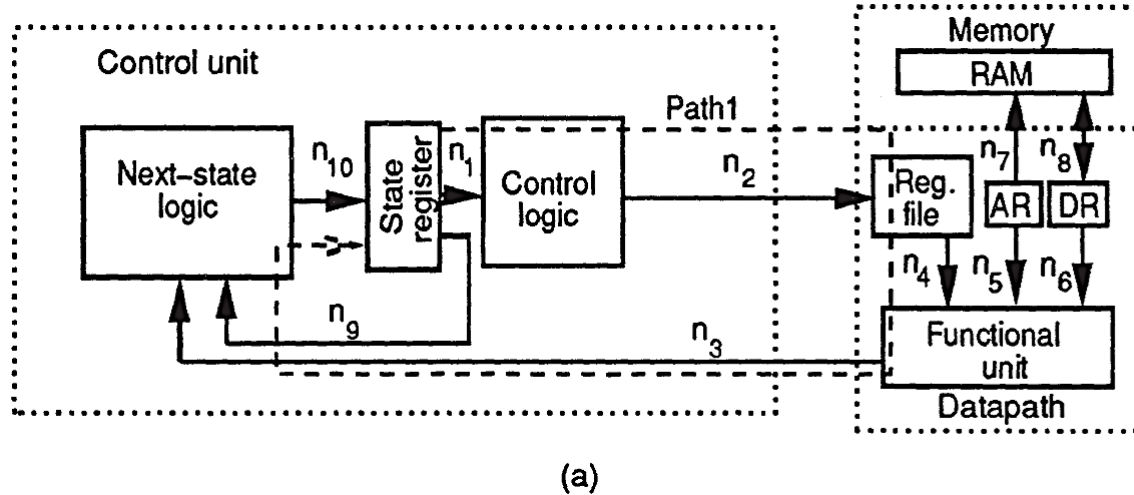
- When the **clock period** for a **FSMD** is computed we must include the delay in the control unit in addition to delays in the datapath.
- A typical FSMD structure is shown in Figure 12.a. There is one critical path (Path1) from the *State* register, through the *Control* logic, the *Reg. file*, the *Functional unit*, the *Next-state logic* and back to the *State register*.
- Thus, the clock period is the sum of all the unit delays and all the net delays, that is,

$$\begin{aligned} t_{\text{clock}} = & t_p(\text{State register}) + t_p(\text{Control logic}) + t_p(\text{Reg. file}) \\ & + t_p(FU) + t_p(\text{Next-state logic}) + t_{\text{setup}}(\text{State register}) \quad (16) \\ & + \sum_{i=1,2,3,4,10} t_p(n_i). \end{aligned}$$

Performance Measures: System Clock Cycle

→ **Figure 12:** System clocking models:

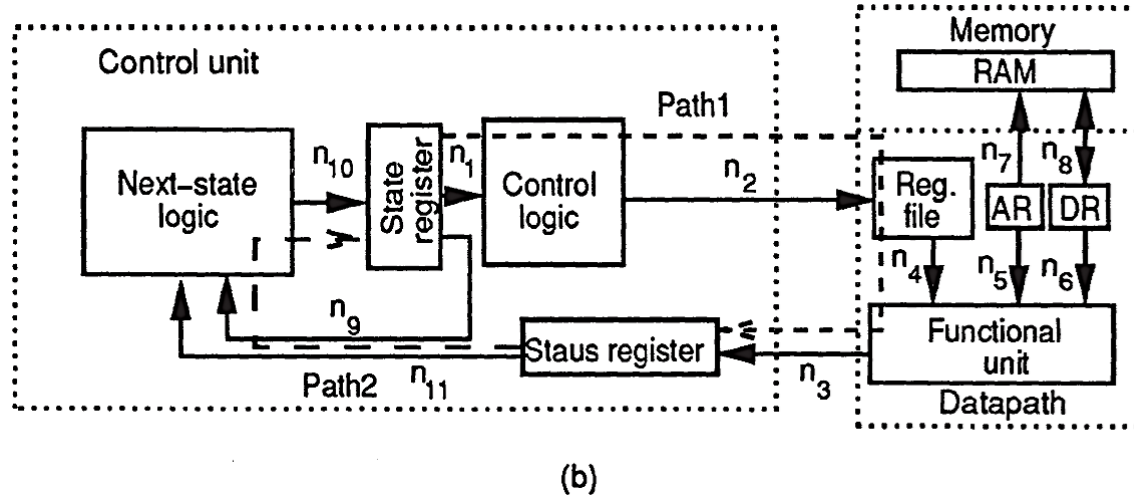
- a) nonpipelined control,
- b) one-stage pipelined control,
- c) two-stage pipelined control.



Performance Measures: System Clock Cycle

→ **Figure 12:** System clocking models:

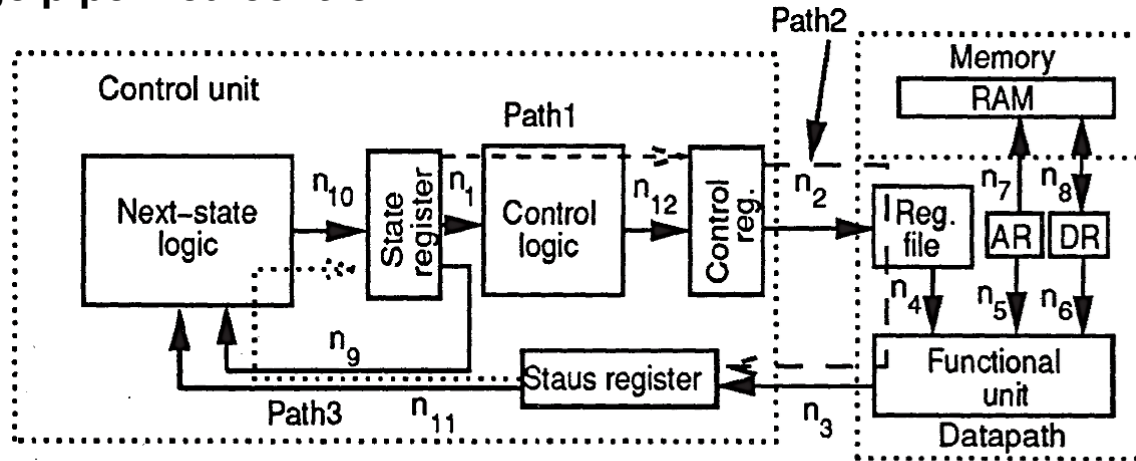
- a) nonpipelined control,
- b) one-stage pipelined control,**
- c) two-stage pipelined control.



Performance Measures: System Clock Cycle

→ **Figure 12:** System clocking models:

- a) nonpipelined control,
- b) one-stage pipelined control,
- c) **two-stage pipelined control.**



(c)

Performance Measures: System Clock Cycle

- The clock period calculated by the Equation 16 may be too long for many applications.
- In practice, the design must be pipelined to reduce the clock period by insertion of the *Status register* (Figure 12.b).
- The clock period is now determined by the longer of two paths, Path1 and Path2.
- Path1 starts with the *State register* and goes through the *Control logic*, the *Reg. file*, the *Functional unit* and ends with the *Status register*, whereas Path2 starts with the *Status register*, goes through the *Next-state logic* and ends in the *State register*. Thus, the clock period is

$$t_{clock} = \text{MAX} \left((t_p(\text{State register}) + t_p(\text{Control logic}) + t_p(\text{Reg. file}) + t_p(\text{FU}) + t_{\text{setup}}(\text{Status register}) + \sum_{i=1,2,3,4} t_p(n_i)), \right. \\ \left. (t_p(\text{Status register}) + t_p(\text{Next-state logic}) + t_{\text{setup}}(\text{State register}) + \sum_{i=10,11} t_p(n_i)) \right). \quad (17)$$

Performance Measures: System Clock Cycle

- The design can be further pipelined by introduction of the *Control register* as shown in Figure 12.c.
- The clock period is then determined by the longest of these three paths *Path1*, *Path2* and *Path3*, that is,

$$\begin{aligned} t_{\text{clock}} = \text{MAX} \quad & ((t_p(\text{State register}) + t_p(\text{Control logic}) \\ & + t_{\text{setup}}(\text{Control register}) + \sum_{i=1,12} t_p(n_i)), \\ & (t_p(\text{Control register}) + t_p(\text{Reg. file}) + t_p(FU) \\ & + t_{\text{setup}}(\text{Status register}) + \sum_{i=2,3,4} t_p(n_i)), \\ & (t_p(\text{Status register}) + t_p(\text{Next-state logic}) \\ & + t_{\text{setup}}(\text{State register}) + \sum_{i=10,11} t_p(n_i))). \end{aligned} \quad (18)$$

Performance Measures: System Clock Cycle

- The **functional unit** (FU) in the **datapath** can be further **pipelined** which makes **clock period** even shorter. The clock period can be estimated using Equations 16,17 and 18.
- The delays of registers, register files, and functional units are known from the given RT component library or predefined component instances.
- The delay of the **control unit** depends on the selected layout style. Using the standard-cell implementation, the propagation delay of next state and control logic can be calculated based on the sum-of-product delay.
- The PLA implementation uses a **2-phase clock**, the input is latched and the AND array is pre-charged during the first phase of the clock, whereas the output is latched during the second phase of the clock.
- The **propagation delay** of the PLA is the **sum of input latch propagation-delay**, AND-OR plane propagation-delay and the setup time and propagation delay of the output latch.
- Using the NOR-NOR implementation of the AND-OR plane, the propagation delay can be approximated with the **propagation delays** of two NOR gates.

Performance Measures: System Clock Cycle

- When a chip consists of several **communicating FSMDs** we must include **delays due to communication**.
- If two FSMDs communicate **asynchronously**, then the clock calculation for each FSMD is independent of the other. On the other hand, communication time must be included when computing execution time for a given behavioral description.
- If several FSMDs use a common clock, then we must include **register-to-register delays** between communicating FSMDs. If such register transfers between two FSMDs are not needed in each control step, the communication between two FSMDs may take several clock cycles and not influence the clock period.
- In either case we must estimate the **wire lengths** between FSMDs. Such an estimate requires the **floorplan** of the entire chip or an estimate of the floorplan.
- If two FSMDs are on different chips, **multi-chip modules** (MCM) or **printed-circuit boards** (PCB), we must estimate pin capacitances and connection delays on the chip, MCM or PCB.



Other Measures

Other Measures

- In a naïve approach, high-level synthesis focused on quality measures based on **area** and **performance** (**speed**).
- Other measures, such as **power consumption**, **testability**, **verifiability**, **reliability**, **manufacturability**, and ... might not be well defined and have not been widely used.
- For example, the **power consumption** of the CMOS digital system consists of two elements:
 - Dynamic dissipation.
 - Static dissipation
- The **dynamic dissipation** of CMOS circuits is caused by **charging** and **discharging** of **load capacitances** during signal transitions.
- When a CMOS gate changes state from either "0" to "1" or "1" to "0", both n- and p-transistors are turned on for a short period of time. This results in a short current pulse to charge and discharge the output capacitive load. One way to compute the dynamic dissipation (P_d) of a CMOS digital system is

$$P_d = P_{gate} \times N \times f_c$$

Other Measures

- **Static power** dissipation in CMOS gates typically arises from two sources:
 - **Leakage Current**: Even when a CMOS gate is **not switching**, there is a tiny amount of current that leaks through the transistors. This is because no semiconductor device can completely block the flow of current, and modern CMOS devices, with their very thin gate oxides and small geometries, can have significant leakage currents. There are several types of leakage, such as gate-oxide leakage, subthreshold leakage, and junction leakage.
 - **Subthreshold Conduction**: When transistors switch from the 'off' state to the 'on' state, there is a region where the transistor is not quite off but not fully on. In this subthreshold region, the transistor can conduct a small amount of current. This is more significant in modern transistors that operate at very low threshold voltages.
- Overall, static power dissipation in CMOS circuits has become a more pressing issue as transistor sizes have shrunk with advances in semiconductor manufacturing technology. These smaller transistors leak more current, and as densities of transistors on chips have increased, even these small leakage currents can add up to significant amounts of power.
- The main strategy to reduce static power dissipation is to improve **transistor design** to **lower leakage currents**, which includes designing **high-k gate dielectrics**, using **multi-threshold voltage processes**, deploying **power gating techniques** to completely shut off power to parts of a chip that are not in use, and improving overall semiconductor manufacturing processes.

Modern Metrics

- **Area:** The area of a design is estimated by the synthesis tool and is specified either as μm^2 or as gate equivalents (GE), where one GE is equivalent to the area of a two-input NAND gate with the lowest driving strength of the appropriate technology.
- **Power:** The power consumption on the gate level is estimated by switching activity using Synopsys Primetime in μW .
- **Operating frequency:** The operating frequency of the design used for synthesis. It is expressed as MHz.
- **Interface:** bits processed per clock cycle
- **Cycles:** Number of clock cycles to complete a phase of algorithm. From the basic metrics, a set of derived metrics are calculated for performance comparison as given.
- **Throughput:** is the sustainable rate at which new output is produced. It is the product of the operating bit and the interface bits produced per cycle. It is expressed as Byte/word/bits per second.
- **Energy/bit:** is the ratio of the total power consumption and the throughput.
- **Area-time:** is the product of the latency per bit generated and the area of the design. The reciprocal metric is **Throughput Per Area Ratio**. TPAR is specifically critical for high performance applications and is maximum at the highest operating frequency.
- **Power-area-time** is a triple product metric that gives a quantitative comparison of all three critical resources of VLSI design, i.e., compactness, throughput and power consumption.
- **latency** is the response time of a design before it becomes functional (initialization latency) which is more critical for RFID applications.
- **Power-latency** is the product of power consumption and latency time.